# EXPERIMENT 7

**Student Name: Dibyanshu kumar** **UID: 24BAI71026**
**Branch: AIT-CSE (AIML)** **Section/Group: 24AIT_KRG-G2**
**Semester: 4**
**Subject Name: Database Management System** **Subject Code: 24CSH-298**

## EXPERIMENT 7

**Design and Performance Analysis of Materialized Views (SanDisk, JTG, PayPal)**

## AIM

**To design and implement a materialized view and to compare and analyze execution time and performance differences between simple views, complex views, and materialized views, thereby understanding their impact on query optimization and system performance.**

---

## OBJECTIVE

1. **To create simple views, complex views, and materialized views.**
2. **To evaluate their performance by comparing query execution time.**
3. **To highlight the advantages of materialized views in enterprise-level applications.**

---

## SOFTWARE REQUIREMENTS

**Database Management System:**
**Oracle Database Express Edition (Oracle XE)**
**PostgreSQL Database**

**Database Administration Tool / Client Tool:**
**Oracle SQL Developer (for Oracle XE)**
**pgAdmin (for PostgreSQL)**

---

## PROBLEM STATEMENT

# EXPERIMENT 7

**In large-scale enterprise systems, frequent execution of complex queries can significantly affect performance. Organizations require optimized mechanisms to improve query response time without compromising data consistency.**

**Materialized views help improve performance by storing precomputed query results, reducing the need for repeated complex calculations.**

---

## PRACTICAL / EXPERIMENT STEPS

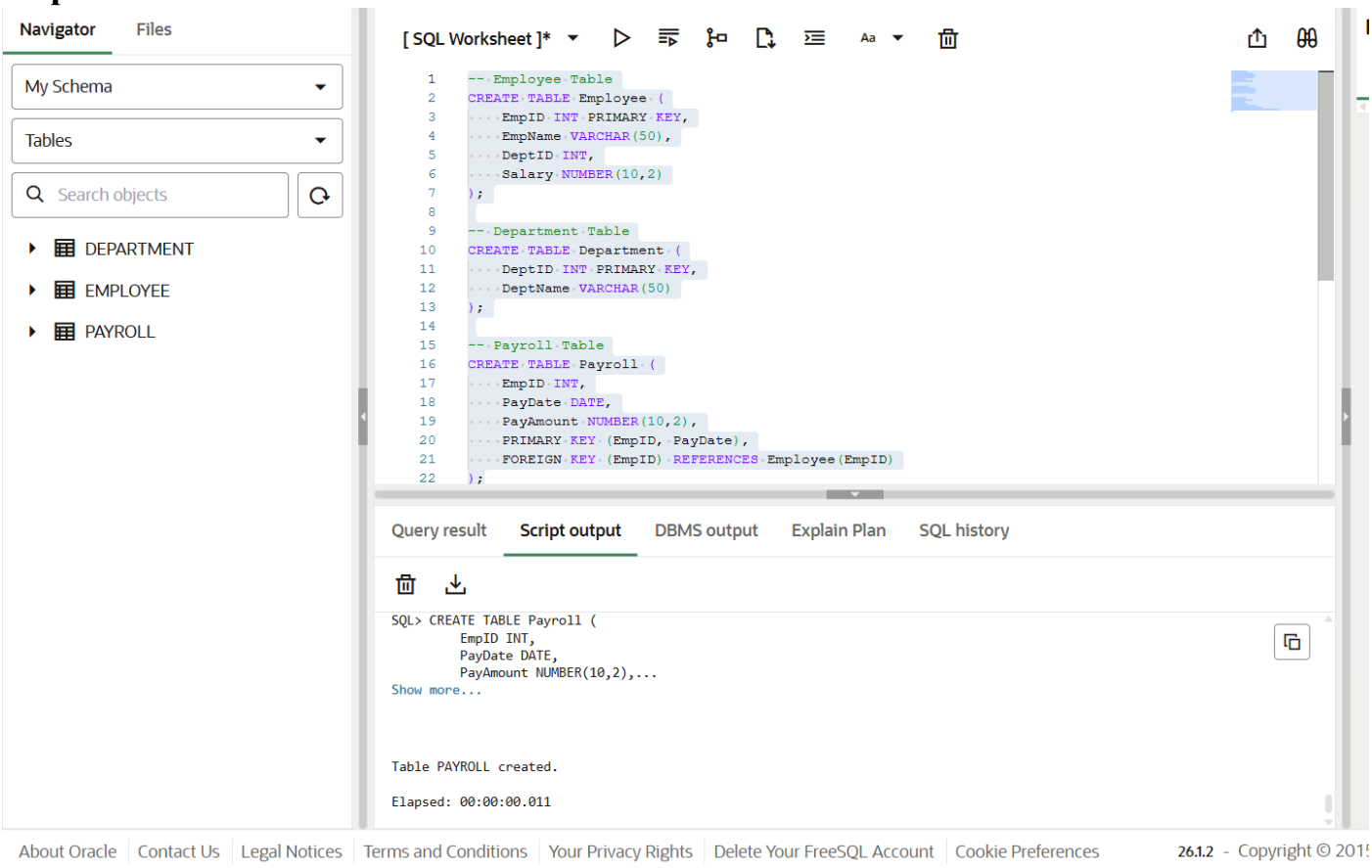
**Step 1: Create base tables (Employee, Department, Payroll).**
**Step 2: Insert sample data into tables.**
**Step 3: Create a simple view based on a single table.**
**Step 4: Create a complex view involving joins and aggregation.**
**Step 5: Create a materialized view storing precomputed results.**
**Step 6: Execute queries on all views.**
**Step 7: Compare execution time and analyze performance.**

---

## STEP-WISE OUTPUT

# EXPERIMENT 7

## Step 1 – Create Base Tables




## Step 2 – Insert Sample Data

# EXPERIMENT 7

Navigator | Files

My Schema

Tables

Search objects

- DEPARTMENT
- EMPLOYEE
- PAYROLL

[ SQL Worksheet ]*

```
--  Departments
INSERT INTO Department VALUES (1, 'IT');
INSERT INTO Department VALUES (2, 'Finance');
INSERT INTO Department VALUES (3, 'HR');

--  Employees
INSERT INTO Employee VALUES (101, 'Alice', 1, 70000);
INSERT INTO Employee VALUES (102, 'Bob', 2, 80000);
INSERT INTO Employee VALUES (103, 'Charlie', 1, 65000);
INSERT INTO Employee VALUES (104, 'David', 3, 60000);

--  Payroll Records
INSERT INTO Payroll VALUES (101, TO_DATE('2026-02-01','YYYY-MM-DD'), 7000);
INSERT INTO Payroll VALUES (102, TO_DATE('2026-02-01','YYYY-MM-DD'), 8000);
INSERT INTO Payroll VALUES (103, TO_DATE('2026-02-01','YYYY-MM-DD'), 6500);
INSERT INTO Payroll VALUES (104, TO_DATE('2026-02-01','YYYY-MM-DD'), 6000);
```

Query result | Script output | DBMS output | Explain Plan | SQL history

```
Elapsed: 00:00:00.002

SQL> INSERT INTO Payroll VALUES (104, TO_DATE('2026-02-01','YYYY-MM-DD'), 6000)

1 row inserted.

Elapsed: 00:00:00.001
```

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account | Cookie Preferences  26.1.2 - Copyright © 20

## Step 3 – Create Simple View

# EXPERIMENT 7

Navigator | Files

My Schema

Tables

Search objects

- DEPARTMENT
- EMPLOYEE
- PAYROLL

[ SQL Worksheet ]*

```sql
INSERT INTO Employee VALUES (102, 'Bob', 2, 80000);
INSERT INTO Employee VALUES (103, 'Charlie', 1, 65000);
INSERT INTO Employee VALUES (104, 'David', 3, 60000);

-- Payroll Records
INSERT INTO Payroll VALUES (101, TO_DATE('2026-02-01','YYYY-MM-DD'), 7000);
INSERT INTO Payroll VALUES (102, TO_DATE('2026-02-01','YYYY-MM-DD'), 8000);
INSERT INTO Payroll VALUES (103, TO_DATE('2026-02-01','YYYY-MM-DD'), 6500);
INSERT INTO Payroll VALUES (104, TO_DATE('2026-02-01','YYYY-MM-DD'), 6000);


CREATE VIEW SimpleEmployeeView AS
SELECT EmpID, EmpName, Salary
FROM Employee;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

```
SQL> CREATE VIEW SimpleEmployeeView AS
     SELECT EmpID, EmpName, Salary
     FROM Employee


View SIMPLEEMPLOYEEVIEW created.

Elapsed: 00:00:00.016
```

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account | Cookie Preferences 26.1.2 - Copyright © 2015,

# EXPERIMENT 7

## Step 4 – Create Complex View

Navigator | Files

My Schema

Tables

Search objects

- DEPARTMENT
- EMPLOYEE
- PAYROLL

[ SQL Worksheet ]*

```sql
CREATE VIEW ComplexEmployeePayroll AS
SELECT e.EmpID, e.EmpName, d.DeptName, SUM(p.PayAmount) AS TotalPaid
FROM Employee e
JOIN Department d ON e.DeptID = d.DeptID
JOIN Payroll p ON e.EmpID = p.EmpID
GROUP BY e.EmpID, e.EmpName, d.DeptName;
```

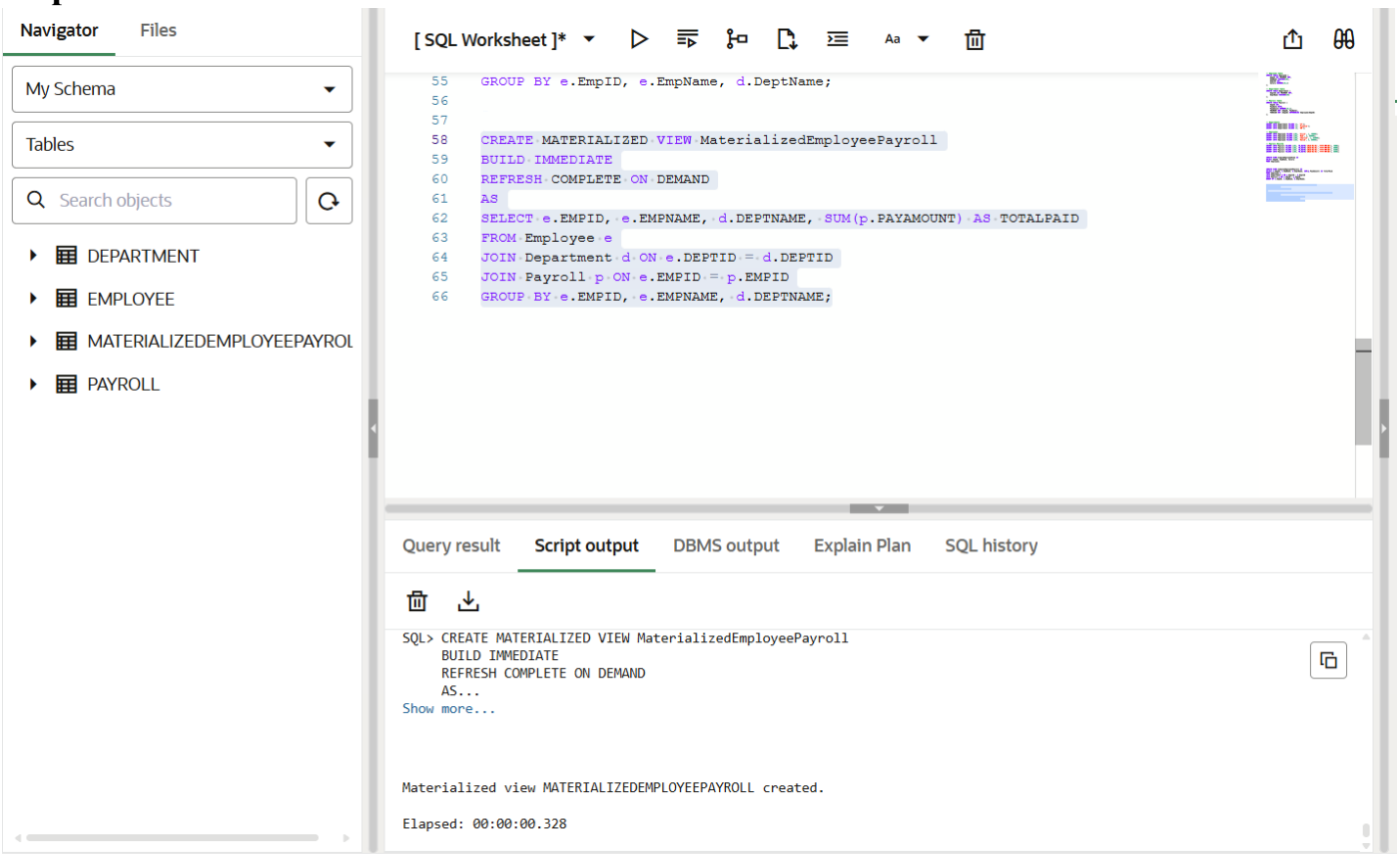
Query result | Script output | DBMS output | Explain Plan | SQL history

```
SQL> CREATE VIEW ComplexEmployeePayroll AS
     SELECT e.EmpID, e.EmpName, d.DeptName, SUM(p.PayAmount) AS TotalPaid
     FROM Employee e
     JOIN Department d ON e.DeptID = d.DeptID...
Show more...


View COMPLEXEMPLOYEEPAYROLL created.

Elapsed: 00:00:00.015
```

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account | Cookie Preferences 26.1.2 - Copyright © 20

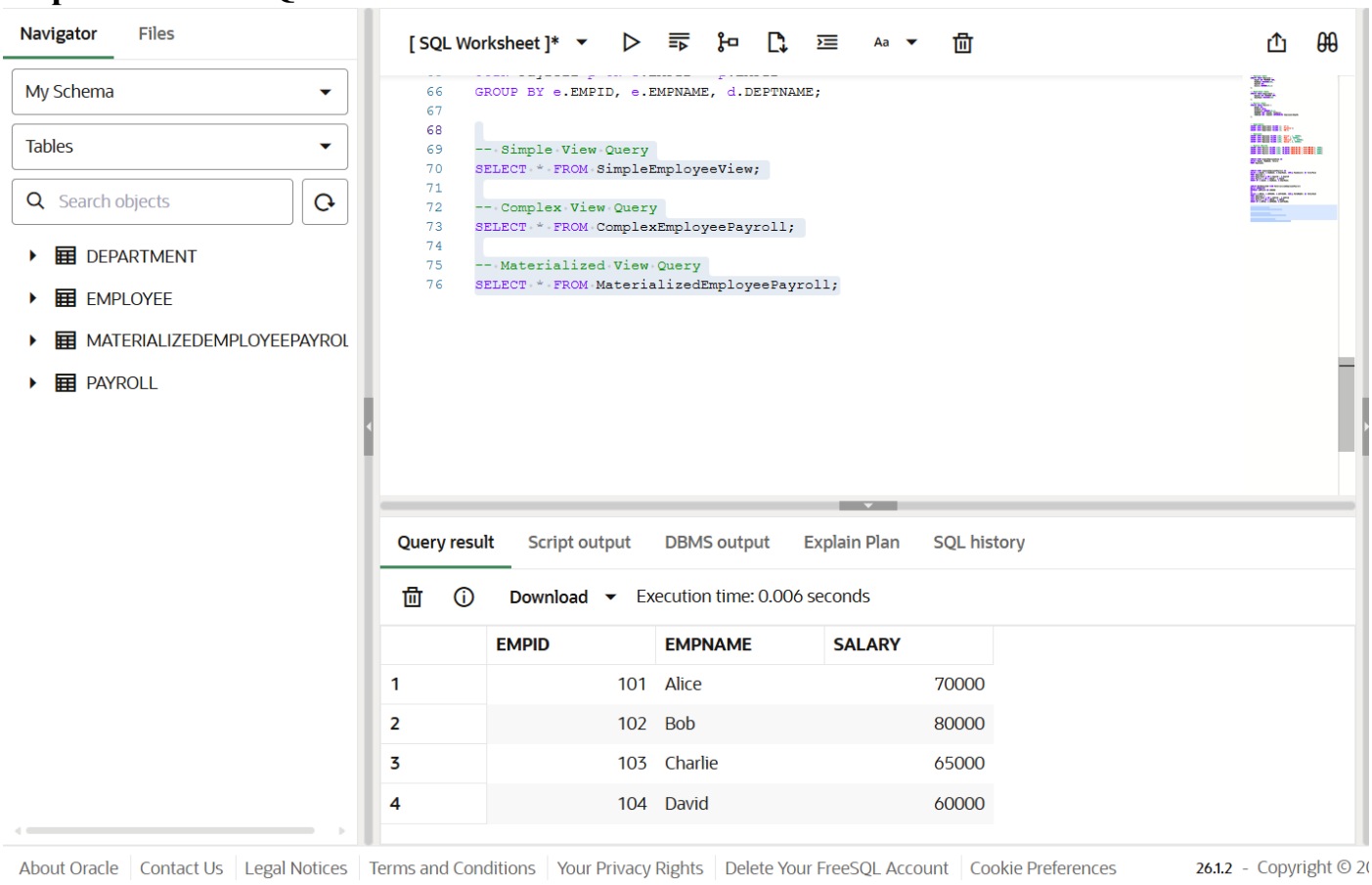
# EXPERIMENT 7

## Step 5 – Create Materialized View



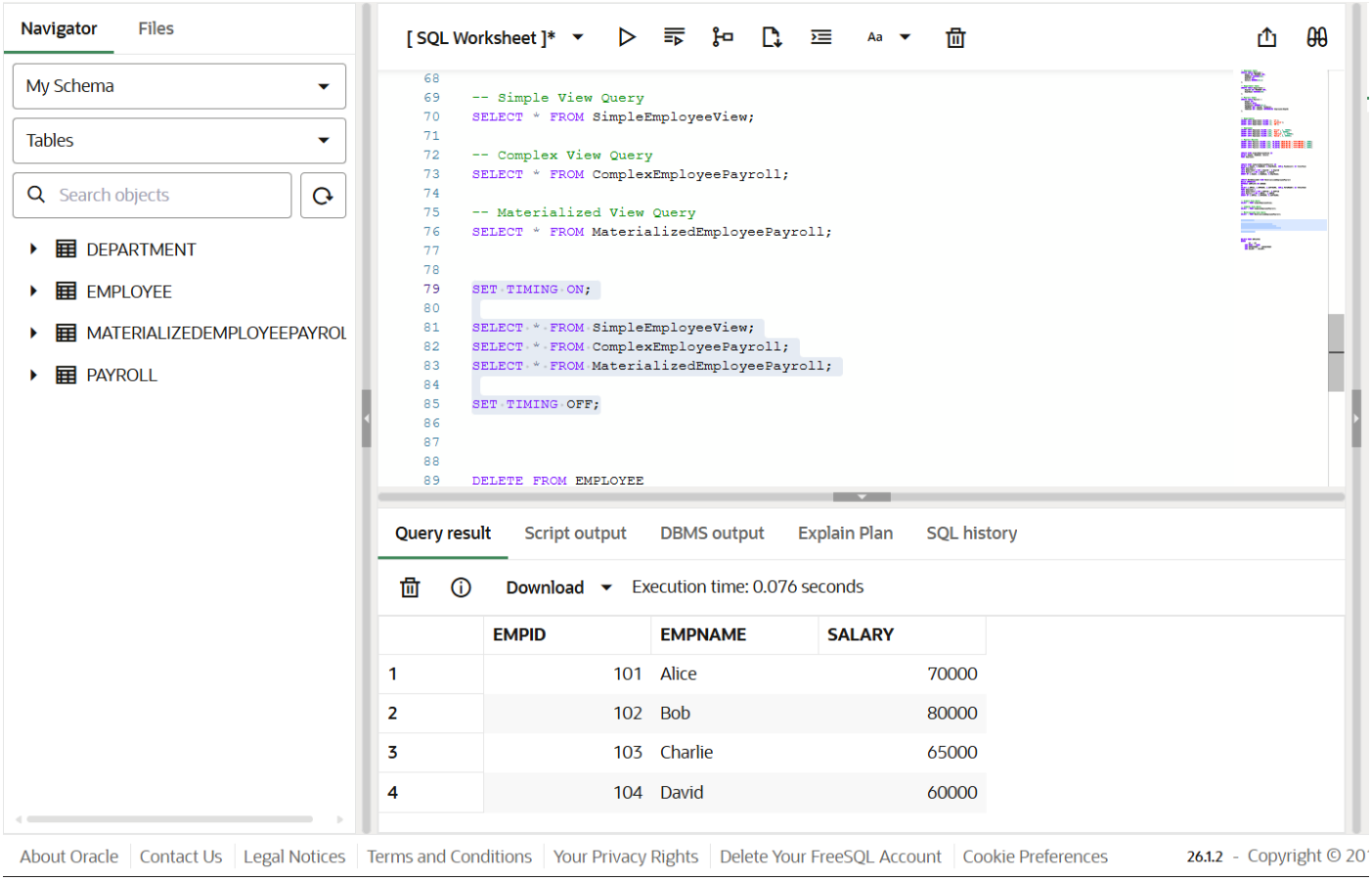
# EXPERIMENT 7

## Step 6 – Execute Queries on Views



# EXPERIMENT 7

## Step 7 – Compare Performance




## RESULT ANALYSIS

1. **Simple View**
   - **Created from a single table.**
   - **Executes quickly for small datasets.**
   - **No performance optimization for complex operations.**
2. **Complex View**
   - **Includes joins and aggregation.**
   - **Slower than simple view because data is computed every time the query runs.**
3. **Materialized View**

- Stores precomputed results physically.
- Significantly faster for repeated queries.
- Requires refresh mechanism to maintain updated data.

---

## REAL-WORLD APPLICATION

**Companies such as SanDisk, JTG, and PayPal handle large-scale transactional data.**

**In such organizations:**

- **Complex reports run frequently.**
- **Data warehouses require fast query responses.**
- **Business intelligence dashboards need optimized performance.**

**Materialized views are used to:**

- **Improve reporting speed.**
- **Reduce server workload.**
- **Enhance query optimization.**
- **Improve overall system performance.**

---

## LEARNING OUTCOME

**After completing this experiment, the learner is able to:**

1. **Understand the concept and working of materialized views in database systems.**
2. **Differentiate between simple views, complex views, and materialized views.**
3. **Create and refresh materialized views in Oracle and PostgreSQL.**
4. **Measure and compare query execution time.**
5. **Analyze performance benefits in data-intensive enterprise applications.**

---

## CONCLUSION

## EXPERIMENT 7

**The experiment demonstrates that materialized views significantly improve query performance for repeated complex operations. While simple and complex views calculate results dynamically, materialized views store results physically, reducing computation time.**

**Therefore, materialized views are highly suitable for enterprise-level systems where performance and efficiency are critical.**